SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
Student Name:	**Kosicram K**	**Reg. No.:**	**192571079**
Date:	**26 October 2026**	**Duration:**	60 minutes

CO2 Statement: *Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*

Code of Conduct

I **Kosicram K / 192571079** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



1. SCENARIO STUDY AND ANALYSIS

Project Title: TestSphere – Integrated Software Testing and Defect Management Platform

1.1 Business Domain and Problem Context

Domain: Software quality assurance and defect management in software development teams. Every release, teams of 5 to 20 members build and test applications, but testing is scattered: test cases live in spreadsheets, bugs are reported in chats and emails, requirements are not linked to tests, and managers see the real quality of a release only at the last moment. TestSphere gives each project one shared platform that makes the testing life cycle and defect life cycle visible, traceable and measurable.

1.2 Existing Challenges

- **Scattered test assets:** test cases and results live in spreadsheets and documents, and changes are never recorded.
- **No execution tracking:** testing progress is unknown until release day.
- **Poor defect tracking:** bugs reported in chats or emails are lost, duplicated or left without an owner.
- **No traceability:** there is no link between a requirement, its test cases and the defects found, so coverage gaps stay hidden.
- **Unclear release quality:** managers lack metrics such as pass rate, open defects and defect density to judge readiness.
- **Slow communication:** fixes and retest requests are exchanged informally with no follow-up.

1.3 System Objectives

1. Give every project one shared workspace for requirements, test cases, test execution and defects.
2. Support structured test planning with test plans, suites, assignments and release cycles.
3. Provide requirement-to-test-to-defect traceability and full version history.
4. Manage the complete defect life cycle from logging to fix, retest and closure.
5. Automate alerts, dashboards and test summary reports with quality metrics.
6. Keep all data secure, available and auditable.

1.4 Stakeholders

Stakeholder	Type	Needs from the system
Tester / QA engineer	Primary user	Write test cases, execute tests, log defects and trace results to requirements.
Test Lead	Primary user	Create projects, plan test cycles, assign test cases, approve requirements and track execution.
Developer	Primary user	See assigned defects with full details, update fix status and link commits and builds.
Project / QA Manager	Primary user	Monitor quality and release readiness across projects, define exit criteria and view reports.
System Administrator	Primary user	Manage accounts, security, backups and the audit log.
Organisation / IT department	Sponsor	Low hosting cost, data security and policy compliance.
Client / Business analyst	Secondary user	Read-only access to test summary and release reports.
Git/CI and email providers	External systems	Supply commit and build data and deliver notification emails.

2. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

2.1 Functional Requirements

36 functional requirements are grouped by module and prioritised with MoSCoW (M = Must, S = Should, C = Could).

ID	Requirement	Pri.
User and access management		
FR-01	The system shall let a new user register with name, organisational email, employee ID and role request.	M
FR-02	The system shall verify the email address before activating an account.	M
FR-03	The system shall authenticate users by email and password, with optional two-factor login.	M
FR-04	The system shall enforce role-based access so each role sees only its permitted functions.	M
FR-05	The administrator shall be able to create, edit, deactivate and assign roles to users.	M
FR-06	The system shall let users reset a forgotten password through a time-limited email link.	M
Project and team management		
FR-07	A Test Lead shall be able to create a project with name, description, modules and dates.	M
FR-08	A Test Lead shall be able to invite testers and developers to a project and assign their roles.	M
FR-09	The Project Manager shall be able to assign a Test Lead to each project and view all projects.	M
FR-10	The system shall support release cycles with phases (Planning, Testing, Regression, Closed).	S
Requirements management		
FR-11	A member shall be able to add a requirement with description, priority and acceptance criteria.	M
FR-12	The system shall keep a version history of every requirement change.	M
FR-13	The Test Lead shall be able to approve a requirement for testing.	S
FR-14	The system shall flag requirements that have no acceptance criteria or use vague words.	C
FR-15	The system shall link each requirement to its test cases and defects (traceability matrix).	M
Test case and execution management		
FR-16	The team shall be able to create test plans and test suites for a release.	M
FR-17	A tester shall be able to write test cases with steps, test data, expected result and priority.	M
FR-18	The system shall provide an execution screen to record each test as pass, fail or blocked.	M
FR-19	A Test Lead shall be able to assign test cases to testers with due dates.	M
FR-20	The system shall show a test execution progress chart for each release.	S
Defect management		
FR-21	A tester shall be able to log a defect with steps to reproduce, severity, priority and attachments.	M
FR-22	The system shall manage the defect life cycle: New, Assigned, In Progress, Fixed, Retested, Closed or Reopened.	M
FR-23	The system shall let users link a Git repository and attach commits and builds to defects.	S
Review, automation and quality		

ID	Requirement	Pri.
FR-24	A Test Lead shall be able to comment on any requirement, test case or defect.	M
FR-25	A Test Lead shall be able to open a review request and record the outcome (approved / changes needed).	M
FR-26	Members shall be able to reply to and resolve review comments.	S
FR-27	A developer shall be able to update a defect with fix details and request a retest.	M
FR-28	Testers shall be able to import and export test cases in CSV or Excel format.	S
Evaluation and reporting		
FR-29	The Test Lead shall be able to define release exit criteria (e.g. no open critical defects, 95% pass rate).	M
FR-30	The system shall calculate defect metrics: open/closed count, aging and defect density per module.	M
FR-31	The system shall report each member's activity (tests executed, defects logged and fixed).	M
FR-32	The system shall generate a test summary report as PDF or Excel.	S
Notifications and administration		
FR-33	The system shall send email and in-app alerts for assignments, due dates (24 h before) and defect status changes.	M
FR-34	The system shall show a role-based dashboard with progress, open defects and upcoming deadlines.	M
FR-35	The administrator shall be able to back up and restore data and view the audit log.	M
FR-36	The system shall write an audit log entry for each create, update, delete and login event.	M

Summary: 28 Must, 7 Should, 1 Could.

2.2 Non-Functional Requirements

Each non-functional requirement carries a measurable target so that it can be tested.

ID	Category	Requirement	Measure / Target
NFR-01	Performance	Pages shall load quickly under normal load.	95% of pages < 3 s with 500 concurrent users
NFR-02	Performance	Search and dashboard queries shall respond fast.	< 2 s for 10,000 test cases and defects
NFR-03	Security	Passwords stored hashed and salted; all traffic encrypted.	bcrypt or stronger; HTTPS (TLS 1.2+)
NFR-04	Security	Access follows RBAC; sessions expire; common web attacks resisted.	30 min idle logout; no critical OWASP Top 10 issues
NFR-05	Reliability	The system shall run without failure in normal use.	MTBF ≥ 500 h
NFR-06	Reliability	Test and defect data shall not be lost on failure.	Daily backup; recovery point ≤ 24 h
NFR-07	Availability	Available during working hours of active projects.	99.5% uptime excluding planned maintenance
NFR-08	Usability	A new tester shall log a defect without training; accessible and responsive.	Log a defect in < 2 min; WCAG 2.1 AA
NFR-09	Scalability	The system shall grow with more projects without redesign.	200 projects (1,000 users) now; 1,000 projects with added servers
NFR-10	Maintainability	Code shall be modular, tested and documented.	ε 70% υνιτ–τεστ χοωεραγε; ΡΕΣΤ ΑΠΙ δοχυ μεντεδ
NFR-11	Portability	The web client shall work on current browsers.	Latest 2 versions of Chrome, Edge, Firefox, Safari
NFR-12	Auditability	Changes to requirements, test results and defect status shall be traceable.	100% of changes logged with user and time
NFR-13	Privacy	Personal and project data used only for authorised purposes.	Compliant with DPDP Act, 2023

3. ACTORS AND USE CASES

3.1 Actors

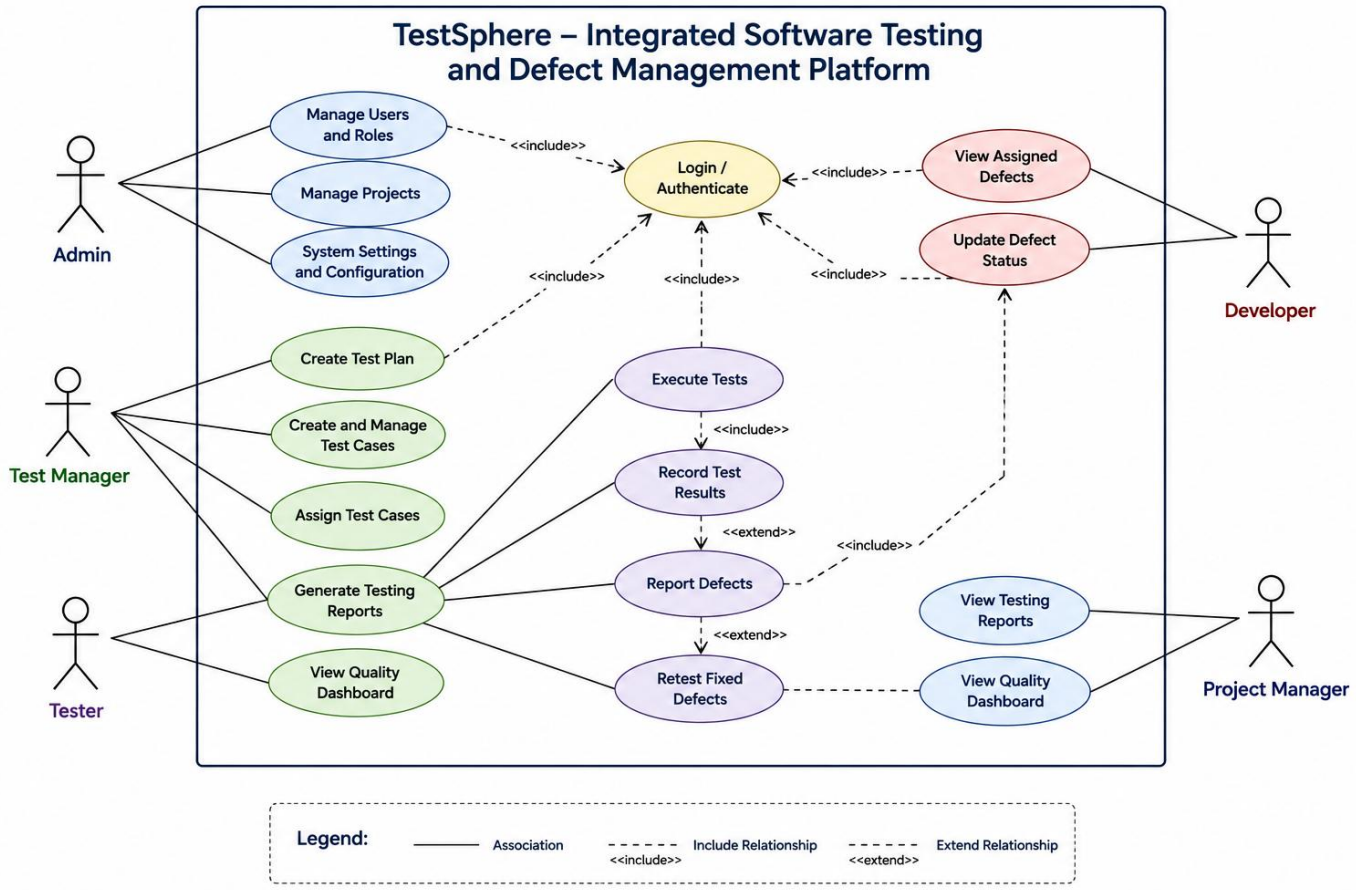
Actor	Type	Description
Tester	Primary	QA engineer who writes test cases, executes tests, logs defects and retests fixes.
Test Lead	Primary	A Tester who also creates projects, plans test cycles, assigns tests and reviews (specialises Tester).
Developer	Primary	Fixes assigned defects, updates their status and links commits and builds.
Project Manager	Primary	Oversees all projects, defines exit criteria and reviews quality metrics and reports.
Administrator	Primary	Manages accounts, backups and the audit log.
Git / CI Service	Secondary	External service (GitHub / GitLab) that supplies commit and build data.
Email Service	Secondary	External SMTP provider that delivers notification emails.

3.2 Major Use Cases by Actor

ID	Use Case	Actor(s)	Related FR
UC-01	Register / Login	All human actors	FR-01 – FR-04, FR-06
UC-02	Manage Users	Administrator	FR-05
UC-03	Create Project & Team	Test Lead	FR-07 – FR-09
UC-04	Plan Test Cycle	Test Lead	FR-16, FR-20
UC-05	Manage Requirements	Tester, Test Lead	FR-11, FR-12, FR-15
UC-06	Create Test Case	Tester, Test Lead	FR-17, FR-28
UC-07	Assign & Schedule Tests	Test Lead	FR-19
UC-08	Execute Test & Record Results	Tester, Test Lead	FR-18
UC-09	Log Defect (extends UC-08)	Tester, Test Lead	FR-21, FR-22
UC-10	Link Git / CI Repository	Developer, Test Lead, Git / CI Service	FR-23
UC-11	View Dashboard & Reports	Tester, Test Lead, Developer, Project Manager	FR-30 – FR-32, FR-34
UC-12	Send Notifications	Email Service (system-triggered)	FR-33
UC-13	Review & Give Feedback	Test Lead	FR-24 – FR-26
UC-14	Approve Requirements	Test Lead	FR-13
UC-15	Define Exit Criteria	Test Lead, Project Manager	FR-29
UC-16	Fix Defect (includes UC-17)	Developer	FR-27
UC-17	Retest & Close Defect	Tester, Test Lead	FR-22
UC-18	Backup & Audit Log	Administrator	FR-35, FR-36

3.3 Use Case Diagram

The diagram follows UML notation: actors outside the system boundary, use cases as ovals, solid lines for associations, a solid arrow for generalisation and dashed arrows for «include» and «extend» relationships.


TestSphere – Integrated Software Testing and Defect Management Platform
Admin
Test Manager
Tester
Developer
Project Manager
Manage Users and Roles
Manage Projects
System Settings and Configuration
Login / Authenticate
View Assigned Defects
Update Defect Status
Create Test Plan
Create and Manage Test Cases
Assign Test Cases
Generate Testing Reports
View Quality Dashboard
Execute Tests
Record Test Results
Report Defects
Retest Fixed Defects
View Testing Reports
View Quality Dashboard
<<include>>
<<extend>>
Legend:
Association
Include Relationship
Extend Relationship

4. SOFTWARE REQUIREMENT SPECIFICATION (SRS)

4.1 Introduction

Purpose: This SRS defines what TestSphere must do and how well it must do it, giving testers, developers, managers and the development team one agreed baseline. **Audience:** test leads, testers, developers, project managers, administrators and quality auditors. **Conventions:** requirements are numbered FR-xx and NFR-xx, use cases UC-xx; priorities follow MoSCoW. **References:** IEEE 830 / ISO/IEC/IEEE 29148; ISTQB Foundation Syllabus; course notes of CSA10 / CSA1063 Software Engineering; Pressman, Software Engineering: A Practitioner's Approach.

4.2 Problem Description

Testing teams manage requirements, test cases, test results and bugs in separate tools, so test coverage is unclear, defects are lost and release quality cannot be measured. TestSphere is a web platform where a team plans test cycles, records versioned requirements linked to test cases and defects, executes tests, logs and tracks defects through their life cycle, receives review feedback, and where leads and managers see live quality metrics and reports to decide release readiness.

4.3 Scope of the System

In scope	Out of scope (this release)
Registration, login and role-based access Project and team creation Requirements with versions and traceability Test plans, suites and test case management Test execution and progress tracking Defect logging, assignment, fix and retest life cycle Git/CI commit and build links, notifications, dashboards, reports, exit criteria	Writing or running automated test scripts (no built-in test automation engine) Native mobile apps (responsive web only) Performance and load test execution tools Payment processing

4.4 Use Case Descriptions

UC-01 Register / Login

Field	Description
Actor	Tester, Test Lead, Developer, Project Manager, Administrator
Precondition	User has an organisational email address.
Main flow	1. User chooses Register or Login. 2. For registration, user enters name, email, employee ID and role; system sends a verification email. 3. User clicks the link and the account becomes active. 4. User enters email and password (and 2FA code if enabled). 5. System validates credentials and opens the role-based dashboard.
Alternate flows	A1: Wrong password – error shown; after 5 failures the account locks for 15 min. A2: Forgotten password – user requests a reset link by email, valid for 30 min.
Postcondition	User has an authenticated session with role-based permissions.

UC-03 Create Project & Team

Field	Description
Actor	Test Lead
Precondition	Test Lead is logged in and has permission to create projects.
Main flow	1. Test Lead selects New Project. 2. Enters name, description, modules, start and end dates. 3. Invites testers and developers by email and assigns roles. 4. System creates the workspace and notifies members. 5. Project Manager reviews the project setup.
Alternate flows	A1: Invited email has no account – system sends a registration invitation. A2: Project name already exists – system asks for a different name.
Postcondition	A project workspace exists with its members and roles.

UC-05 Manage Requirements

Field	Description
Actor	Tester, Test Lead (Test Lead approves via UC-14)
Precondition	Project exists; user is a member.
Main flow	1. Member chooses Add Requirement. 2. Enters description, type, priority and acceptance criteria. 3. System saves it as version 1. 4. Member links it to test cases and defects. 5. Member submits it for Test Lead approval.
Alternate flows	A1: Acceptance criteria empty – system warns the requirement is ambiguous. A2: Editing an approved requirement – system creates a new version as Draft.
Postcondition	Requirement is stored with full version history and traceability links.

UC-04 Plan Test Cycle

Field	Description
Actor	Test Lead
Precondition	At least one approved requirement exists.
Main flow	1. Test Lead creates a test plan for the release. 2. Groups test cases into test suites. 3. Sets the cycle name, start and end dates. 4. Assigns test cases to testers with due dates. 5. System opens the execution board and progress chart.
Alternate flows	A1: Tester workload exceeds capacity – system warns the Test Lead.
Postcondition	An active test cycle with assigned test cases exists.

UC-13 Review & Give Feedback

Field	Description
Actor	Test Lead
Precondition	A requirement, test case or defect has been submitted for review.
Main flow	1. Test Lead opens the review request. 2. Reads the item and adds comments. 3. Marks the outcome Approved or Changes Needed. 4. System notifies the author. 5. Author replies and resolves comments.
Alternate flows	A1: Reviewer does not respond within 3 days – system reminds the reviewer and informs the Project Manager.
Postcondition	Review outcome and comments are stored against the item.

UC-16 Fix Defect

Field	Description
Actor	Developer, Tester
Precondition	Defect is assigned to the developer with status Assigned.
Main flow	1. Developer opens the defect and reads the steps to reproduce. 2. Sets status In Progress. 3. Fixes the code and links the commit or build. 4. Marks the defect Fixed and requests retest. 5. System notifies the tester, who retests (UC-17).
Alternate flows	A1: Defect cannot be reproduced – developer returns it to the tester with a comment. A2: Retest fails – defect is Reopened and returned to the developer.
Postcondition	Defect is Fixed (Closed after a successful retest) and the change is logged in the audit trail.

4.5 Data Requirements

Entity	Key attributes	Relationships
User	user_id (PK), name, email (unique), password_hash, employee_id, role, status	A user belongs to many projects
Project / ProjectMember	project_id (PK), name, description, lead_id (FK), start_date, end_date; member_id, role	Many requirements, releases and members
Release	release_id (PK), project_id (FK), name, phase, start_date, end_date	Many test suites and defects
Requirement	req_id (PK), project_id (FK), type, description, priority, acceptance_criteria, status, version	Many versions, test cases and defects
TestSuite / TestCase	suite_id (PK), release_id (FK); test_id (PK), req_id (FK), steps, expected_result, priority	A suite has many test cases
TestExecution	exec_id (PK), test_id (FK), assignee_id (FK), result, executed_at, due_date	One test case, many executions
Defect	defect_id (PK), exec_id (FK), req_id (FK), title, severity, priority, status, assignee_id (FK)	One execution may raise many defects
Attachment / Comment	file_id (PK), defect_id (FK), file_path, size_kb; comment_id (PK), user_id (FK), text	A defect has many attachments and comments
Review	review_id (PK), target_type, target_id, reviewer_id (FK), outcome, created_at	Many comments
ExitCriteria	criteria_id (PK), release_id (FK), metric, threshold	Many criteria per release
AuditLog	log_id (PK), user_id (FK), action, entity, timestamp, ip_address	Written on every create, update, delete and login

- Email and employee ID are unique per user. Requirement and defect changes keep earlier versions; nothing is overwritten.
- Project data is kept for 5 years after release closure; audit logs for 2 years. User accounts are deactivated, not deleted, so defect history stays intact.
- Attachments are limited to 25 MB and to PNG, JPG, PDF, TXT and LOG types. Estimated storage is about 4 GB per term for 200 projects.

4.6 External Interface Requirements

Interface	Requirement
User interface	Responsive web interface with a role-based dashboard, left navigation menu and top bar with notifications. Main screens: login, dashboard, projects, requirements, test cases, test execution, defects, reviews, reports and admin console. Keyboard navigation and screen-reader support (WCAG 2.1 AA).
Hardware	Any device with a modern browser. Server: Linux VM with at least 4 vCPU, 8 GB RAM and 200 GB storage.
Software	PostgreSQL database; object storage or file system for attachments; Git hosting / CI API (GitHub / GitLab, OAuth, REST/JSON); SMTP for email; PDF generator for reports; optional LDAP/OAuth directory to verify organisational emails.
Communication	All client-server traffic over HTTPS (TLS 1.2+). Internal REST API with JSON under /api/v1 protected by session tokens. Emails sent asynchronously through a queue.

4.7 System Features

#	Feature	Description	Priority
1	Secure access	Registration, login, 2FA, role-based permissions	High
2	Project workspace	Shared space with members, roles and release cycles	High
3	Requirements and traceability	Versioned requirements linked to test cases and defects	High
4	Test planning and execution	Test plans, suites, assignments, execution and progress chart	High
5	Git / CI and data exchange	Commit and build links, CSV / Excel import and export	Medium
6	Defect management and review	Defect life cycle, attachments, threaded comments, approvals	High
7	Quality metrics and reporting	Defect metrics, exit criteria and test summary reports	High
8	Notifications, dashboards, administration	Alerts, role dashboards, backup, audit log	Medium

5. ASSUMPTIONS AND CONSTRAINTS

5.1 Assumptions

- Every user has an organisational email address and a device with a modern browser and internet access.
- Each project has 5 to 20 members and at least one Test Lead.
- Teams keep their code on a Git hosting service; TestSphere only reads commit and build data from it.
- The organisation provides a server or cloud account and an SMTP service.
- The Project Manager defines severity and priority scales and exit criteria before testing starts.
- Teams follow an iterative (Agile) process with release cycles of 1 to 4 weeks.
- The interface is English only in this release.

5.2 Constraints

Type	Constraint
Technical	Web-based only; open-source stack (PostgreSQL, Linux); must work on current versions of four major browsers; external integrations limited to Git/CI hosting APIs and SMTP.
Operational	Used mainly during active release cycles with peaks before release dates; needs a trained administrator; maintenance outside release windows.
Economic	Budget limited to organisational resources; no paid licences; a single small cloud instance in year one.
Legal	Digital Personal Data Protection Act, 2023; project data used only for authorised purposes; open-source licences respected; WCAG 2.1 AA accessibility.
Time	Prototype in 8 weeks, full release in one semester (about 16 weeks), ready before the next release cycle.
Organisational	Defect life cycle and approval workflow must match the organisation’s QA policy.

6. REQUIREMENT VALIDATION

6.1 Coverage of Stakeholder Requirements

Stakeholder need	Objective	Covered by
Test assets recorded and changes tracked	1, 3	FR-11 – FR-15, FR-17; UC-05, UC-06
Test planning and execution progress visible	2, 5	FR-16 – FR-20, FR-33, FR-34; UC-04, UC-07, UC-08, UC-12
Defects logged, assigned and followed to closure	4	FR-21 – FR-23, FR-27; UC-09, UC-16, UC-17
Managers can judge release quality	5	FR-29 – FR-32, FR-34; UC-11, UC-15
Requirement coverage is traceable	3	FR-12, FR-15; UC-05
Review feedback is recorded and followed up	1, 4	FR-24 – FR-26; UC-13
Security, availability, auditability	6	FR-35, FR-36; NFR-03 – NFR-07, NFR-12; UC-18

6.2 Quality Checks and Findings

Check	Finding	Improvement made / suggested
Ambiguity	NFR-01 and NFR-02 describe performance with “quickly” and “fast”; FR-14 does not define “vague words”.	Each has a measure column as the test; FR-14 uses a configurable list of words such as some, fast, etc.
Inconsistency	Deleting users (FR-05) conflicts with keeping defect history for 5 years.	FR-05 changed to deactivate; the name stays on old defects and personal data is anonymised on request.
Inconsistency	NFR-09 sizes the system for 200 projects but attachment storage was first estimated for 100.	200 projects used everywhere; storage about 4 GB per term.
Redundancy	FR-12, FR-22 and FR-36 all record change history.	Kept: FR-36 is the security trail; FR-12 and FR-22 are user-visible requirement and defect history.
Missing	No export of a project at release closure.	Suggested FR-37: Test Lead exports requirements, test cases, results and defects as CSV / PDF.
Missing	No profile management or notification preferences.	Suggested FR-38: user edits profile and chooses which alerts arrive by email.
Missing	FR-10 (release cycles) has no use case.	Suggested UC-19 Change Release Phase for the Test Lead.

6.3 Conclusion

Every stakeholder need in Section 1 is traced to at least one functional requirement and one use case, except FR-10 (to be covered by UC-19). Every non-functional requirement has a measurable target, and assumptions and constraints are stated. The Git/CI integration (third-party API limits) and the 99.5% availability target (institution’s hosting) should be confirmed before design begins.

End of Document — This submission addresses all six required aspects of the Scenario-Based Requirement Analysis: scenario study, functional and non-functional requirements, actors and use cases with diagram, the SRS, assumptions and constraints, and requirement validation.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	**/25**